CHEAT SHEET · JS INTERVIEW #05 @specsycoder

Promise.all

Run independent requests in parallel

slow.js

```
const user = await getUser();
const orders = await getOrders();
const notes = await getNotifications(); // ~3s
```

fast.js

```
const [user, orders, notes] = await Promise.all([
  getUser(),
  getOrders(),
  getNotifications(),
]); // ~1s
```

output 3s → 1s

The 4 ideas

01 Sequential await
Each await waits for the previous one. Total time = sum of all.

02 Parallel
Promise.all starts them together. Total time ≈ the slowest one.

03 Order kept
Results come back in the same order as the input array.

04 Fails fast
Rejects as soon as any one promise rejects.

Step by step

1. Calling the 3 functions starts all 3 requests immediately.
2. `Promise.all` waits until every one resolves.
3. Destructure the results in order: `[user, orders, notes]`.

Save this · share with a friend Follow @specsycoder for daily JS interview Qs

@specsycoder

all vs allSettled vs race

Pick the right combinator

method	resolves when	rejects when
`Promise.all`	all fulfil (array of values)	first rejection
`Promise.allSettled`	all settle (status objects)	never
`Promise.race`	first to settle fulfils	first to settle rejects
`Promise.any`	first to fulfil	all reject (AggregateError)

settled.js

```
const results = await Promise.allSettled([
  getUser(), getOrders(),
]);
// [{ status, value }, { status, reason }]
```

Traps to mention

- **Only for independent requests.** If orders needs `user.id`, await the user first.
- **await inside forEach does not wait.** Use `await Promise.all(items.map(fn))`.
- **Rejection does not cancel the others.** The remaining requests keep running.

Your 20-second answer

"Awaiting requests one by one makes the total time the sum of all of them. If they are independent, I start them together and await Promise.all, so the total time is roughly the slowest one. If one failing should not break the rest, I use Promise.allSettled."